

Sistema de Gestão Comercial para Pequenos Negócios (SGC)

O projeto consiste no desenvolvimento de um **Sistema de Gestão Comercial** aplicado a um contexto real de comércio (ex: loja de informática, papelaria, livraria, loja de roupas ou assistência técnica).

Formato das Equipes

- Até 4 alunos

Requisitos Arquiteturais Obrigatórios

O projeto deve obrigatoriamente conter:

- Arquitetura em Camadas
- Comunicação com Banco de Dados
- API REST
- Autenticação baseada em Token (JWT)
- Interface gráfica Web (Django)
- Versionamento no GitHub
- Documentação técnica detalhada
 - Casos de Uso
 - Diagrama de Domínio
 - Diagrama de Classes
 - Diagrama lógico do Banco de Dados
 - Manual do Usuário

Requisito de Segurança

Implementar:

- Autenticação via Token (JWT)
- Endpoint de login
- Proteção de rotas
- Controle de acesso por perfil (ADMIN, FUNCIONARIO)

Funcionalidades Obrigatórias

1. Gestão de Clientes

Dados:

- id
- nome
- cpf
- email
- telefone
- endereço

Regras:

- CPF não pode ser duplicado
- Email deve ser válido
- Cliente não pode ser removido se possuir vendas registradas

2. Gestão de Produtos

Dados:

- id
- nome
- descrição

- preço
- quantidade em estoque

Regras:

- Preço não pode ser negativo
- Estoque mínimo deve ser controlado
- Não permitir venda se estoque insuficiente

3. Registro de Venda

Dados:

- id
- data
- cliente
- lista de itens
- valor total
- usuário responsável

Regras:

- Valor total calculado automaticamente
- Atualizar estoque após venda
- Não permitir venda sem itens

4. Autenticação

Dados:

- id
- username
- senha (criptografada)
- perfil

Regras:

- Senha criptografada com BCrypt ou outra forma de criptografia
- Token com tempo de expiração

5. Relatórios**Dados:**

- Produto
- Período
- Cliente

Regras:

- Informar as vendas por período
- Informar vendas do cliente
- Construir representação gráfica das vendas anuais

API REST Obrigatória

Exemplos de endpoints:

POST /auth/login

Input: login e senha

Output: token

GET /clientes

Input:

Output: listagem de clients JSON

POST /clientes

Input: JSON com dados do cliente

Output: JSON com informação de sucesso ou erro na inclusão

PUT /clientes/{id}

Input: JSON com dados do cliente

Output: JSON com informação de sucesso ou erro na alteração

DELETE /clientes/{id}

Input: id do cliente

Output: JSON com informação de sucesso ou erro na exclusão

GET /produtos

Input:

Output: listagem de produtos em JSON

POST /produtos

Input: JSON com dados do produto

Output: JSON com informação de sucesso ou erro na inclusão

PUT /produtos/{id}

Input: JSON com dados do produto

Output: JSON com informação de sucesso ou erro na alteração

DELETE /produtos/{id}

Input: id do produto

Output: JSON com informação de sucesso ou erro na exclusão

GET /vendas

Input:

Output: listagem de vendas

GET /vendas/{id}

Input: id da venda

Output: JSON com informação da venda

POST /vendas

Input: período inicial e final da venda

Output: JSON com listagem das vendas

Deve retornar JSON em qualquer solicitação de endpoints.

Interface (Web)

A aplicação deve permitir:

- Login
- Cadastro de clientes
- Cadastro de produtos
- Registro de vendas
- Listagem de dados
- Relatório de vendas por período

A interface deve consumir a API REST.

Banco de Dados

Deve conter no mínimo as seguintes tabelas:

- usuarios
- clientes
- produtos
- vendas
- itens_venda

Deve ser entregue:

- Script SQL
- Diagrama lógico do banco

Casos de Uso, Diagrama de Domínio e Diagrama de Classes (Obrigatório)

Os alunos devem entregar os diagramas solicitados e os casos de uso associados às funcionalidades implementadas.

Documentação Obrigatória

Deve conter:

1. Descrição da Arquitetura
2. Padrões de Projeto utilizados
3. Justificativa das decisões técnicas
4. Diagrama de Domínio
5. Diagrama de Classes
6. Diagrama lógico do banco
7. Instruções para executar o sistema
8. Link do repositório GitHub

ENTREGAS

ENTREGA 1 – Modelagem e Arquitetura (03/04)

Deve conter:

- Documento PDF com:
 - Descrição do sistema
 - Requisitos funcionais
 - Requisitos não funcionais
 - Arquitetura proposta
 - Padrões de projeto escolhidos
- Diagrama de Classe

- Diagrama de Domínio
- Diagrama lógico do banco
- Script de Criação do Banco de Dados
- Repositório GitHub criado

Avaliação:

Foco em modelagem e arquitetura.

Critério	Descrição do que será avaliado	Pontos
Compreensão do problema	Clareza na descrição do contexto real (comércio/serviço) e dos objetivos do sistema	0,5
Requisitos funcionais e não funcionais	Definição correta e completa das funcionalidades principais; Identificação de aspectos como segurança, desempenho, manutenibilidade.	0,5
Diagrama de Domínio	Coerência das entidades, atributos e relacionamentos	0,5
Diagrama de Classes	Descrição das classes necessárias	0,5
Diagrama lógico do banco	Normalização, chaves primárias e estrangeiras corretas	0,5
Organização inicial no GitHub	Repositório criado, README inicial e commits organizados	0,5

Total: 3,0

r

ENTREGA 2 – Backend e API (08/05)

Deve conter:

- Registro de vendas funcionando
- Controle de estoque
- Relatórios de Vendas

- Banco integrado
- CRUD de clientes e produtos
- Testes básicos
- README atualizado

Avaliação:

Foco em arquitetura, organização do código e funcionamento da API.

Critério	Descrição do que será avaliado	Pontos
Funções de vendas e relatórios	Endpoints funcionais, uso correto de HTTP, JSON e REST	0,6
Integração com banco de dados	Persistência funcional	0,5
Navegabilidade das Telas	Conexão lógica e funcional das telas	0,6
Tratamento de exceções	Uso de exceções personalizadas e handler global	0,4
Organização e legibilidade do código	Código limpo, nomes adequados e estrutura coerente	0,5
Versionamento no GitHub	Commits frequentes, mensagens claras, histórico coerente	0,4

Total: 3,0

ENTREGA 3 – Sistema Completo (26/06)

Deve conter:

- Sistema totalmente funcional
- API REST funcional
- API protegida com JWT
- Recuperação de senha por e-mail
- Documentação final completa

- Projeto versionado corretamente no GitHub

Avaliação:

Foco na integração, funcionamento completo e qualidade arquitetural.

Critério	Descrição do que será avaliado	Pontos
Integração Interface + API	Interface consumindo corretamente a API REST; Usabilidade, organização visual e fluxo funcional	0,6
E-mail	Recuperação de senha através do e-mail	0,5
Regras de negócio	Validações e regras corretamente implementadas	0,5
Funcionamento geral	Sistema completo, sem erros críticos, fluxo consistente	0,5
Documentação final	Documentação clara, completa e atualizada	0,4
Identidade Visual	Sistema com nome e com esquema de cores próprios	0,5
Entrega Apresentação +	Organização final, README completo e instruções de execução	1,0

Total: 4,0